

Computer Science & IT

Theory of Computation



Runtime Environments (One Shot)



By- Ravindra Sir



Recap of Previous Lecture



Topic

Parsem

Topic

Topic

$E \rightarrow E + T / T$
 $T \rightarrow T \times F / F$
 $F \rightarrow id$

$$n(\underline{SLR(I)}) = n_1, \quad n(\underline{LALR(I)}) = n_2 \quad n(CL(I)) = n_3$$

$$\underline{n_1 = n_2} \leq \underline{n_3}$$

$S \rightarrow cC$

$C \rightarrow cC / \underline{d}$

	c	d	\$
S	$S \rightarrow cC$		
C	$C \rightarrow cC$	$C \rightarrow d$	

a) LL(1)

b) SLR(1) but not LL(1)

c) LALR(1) but not SLR(1)

d) CLR(1) but not LALR(1)

Find the no of SR and

$S \rightarrow SS/a/\epsilon$

items with LR(0) item



1SR

1SR

1RR

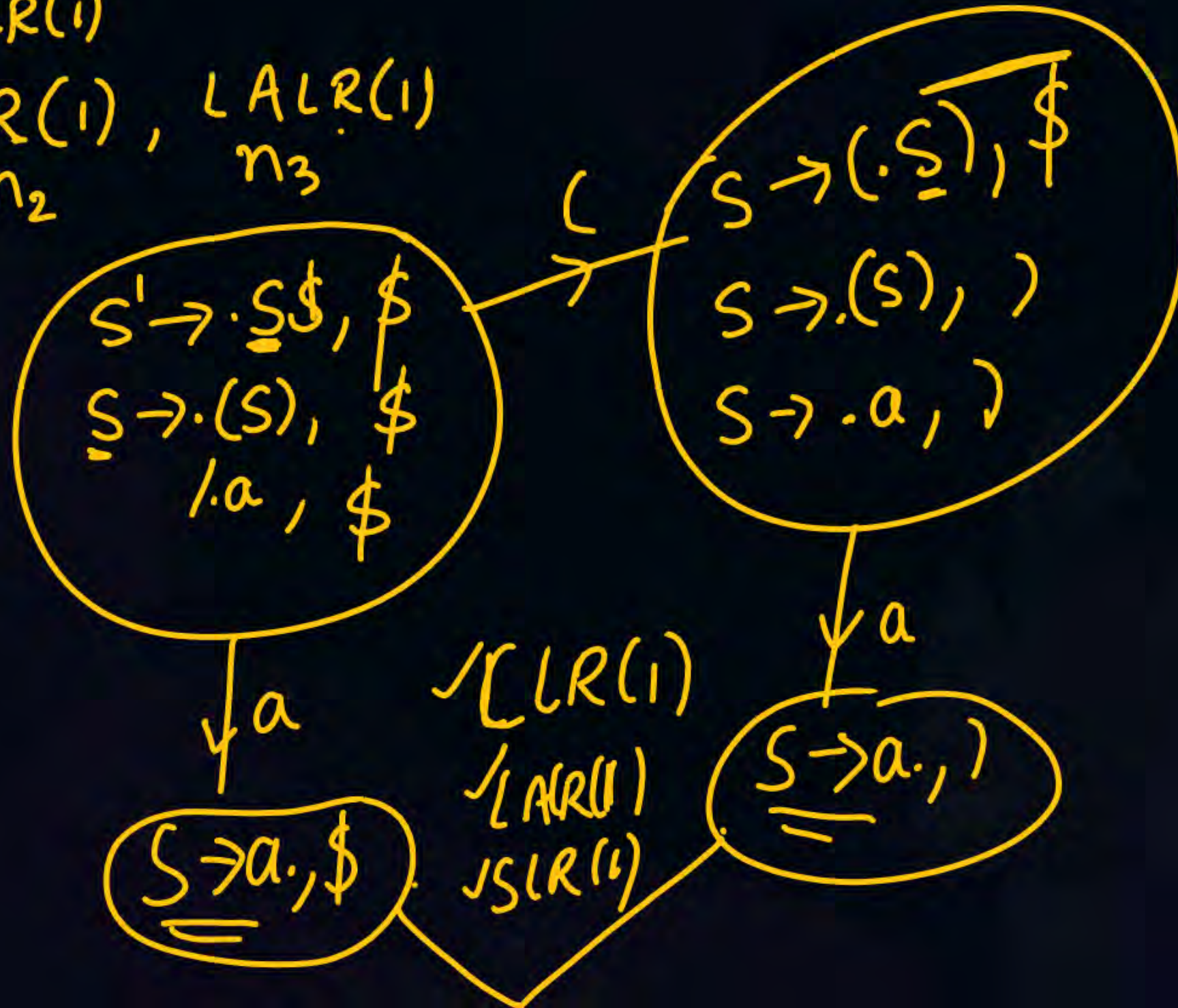
2SR

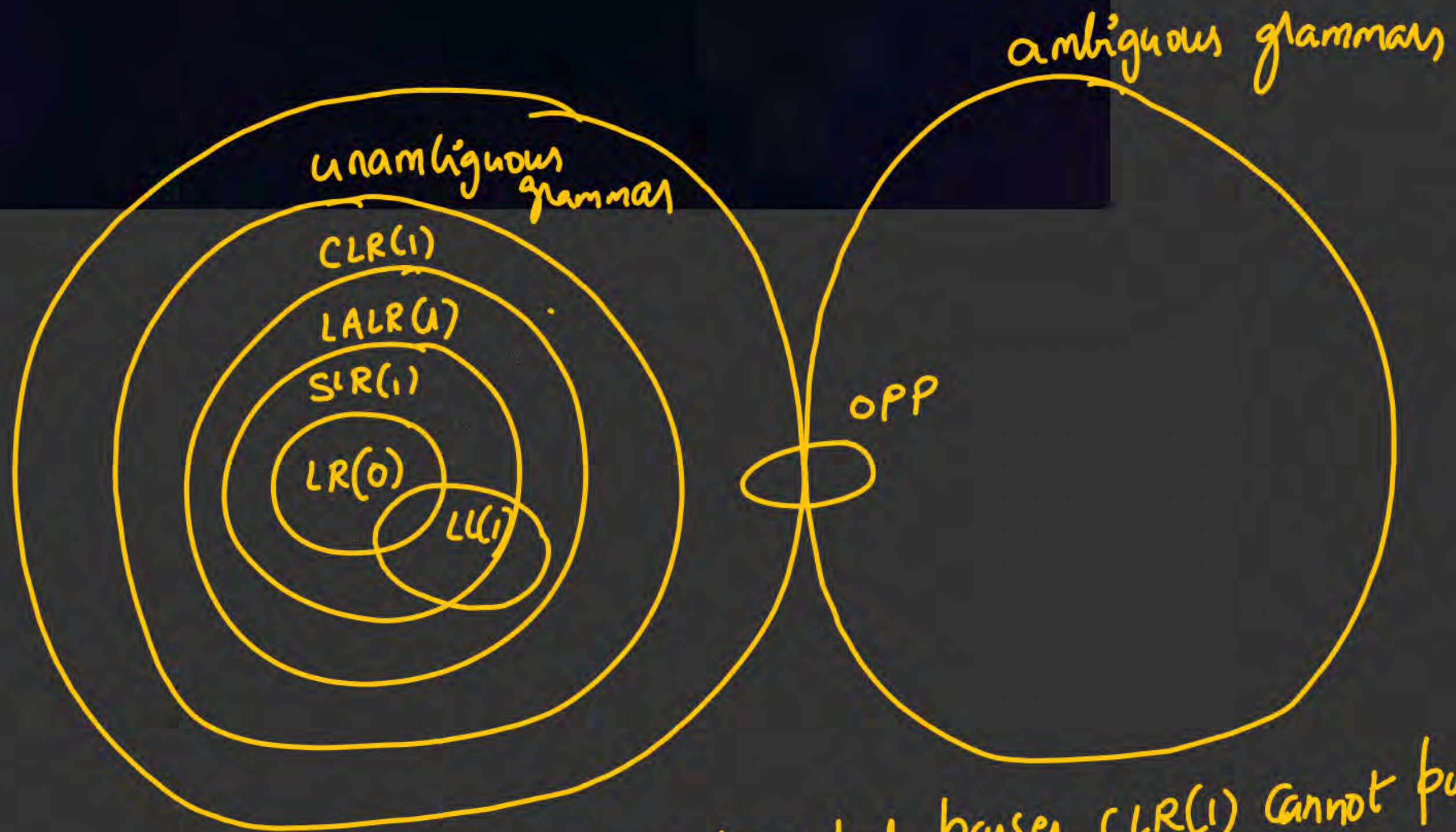
- a) 1SR, 2RR b) 2SR, 2RR c) 4SR, 1RR d) none

$$\frac{S \rightarrow (S)}{/a}$$

$SLR(1)$, $LR(1)$, $LALR(1)$
 n_1 n_2 n_3

- a) $n_1 < n_2 < n_3$
- b) $n_1 = n_3 < n_2$
- c) $n_1 = n_2 = n_3$
- d) $n_1 \geq n_3 \geq n_2$





Even the most powerful parser CLR(1) cannot parse all unambiguous grammars

RBR table

1 Δ 2



	LL(1)	LR(0)	SLR(1)	LALR(1)	CLR(1)
1. $S \rightarrow a$	✓	✓	✓	✓	✓
2. $E \rightarrow E + T / T$ $T \rightarrow a$	x	✓	✓	✓	✓
3. $E \rightarrow E + T / T$ $T \rightarrow T * F / F$ $F \rightarrow id$	x	x	✓	✓	✓
4. $S \rightarrow \underline{A}a / bAC / dC / bda$ $A \rightarrow d$	x	x	x	✓	✓
5. $S \rightarrow Aa / bAC / BC / bBa$ $A \rightarrow d, B \rightarrow d$	x	x	x	x	✓
6. $S \rightarrow a / A$ $A \rightarrow a$	x	x	x	x	x

Comparison of parsers: TDP vs BUP:

Out of all TDPs, most widely used one is LL(1) parser

Out of all BUPs, most widely used one is LALR(1)

Lets compare LL(1) & LALR(1)

Design:- TDP is simpler to construct than BUP

Table:- LALR(1) parsing table is exponential to the

Size of the grammar. This is not the case with LL(1).

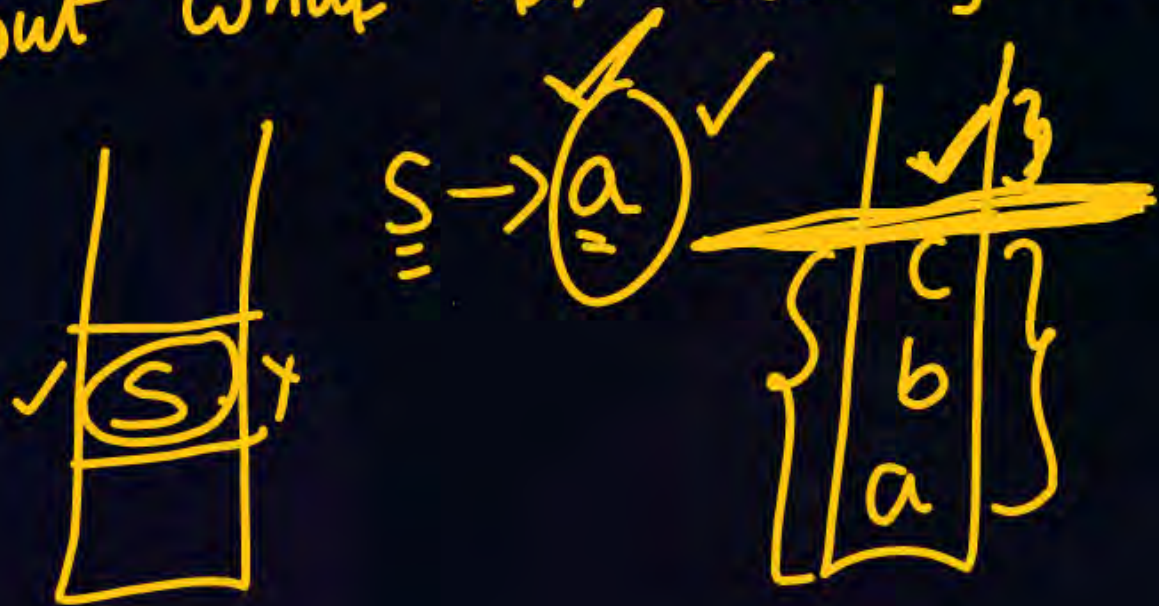
LL(1) size = $O(n^2)$ ✓. The size of BUP is roughly double

the size of TDP.

$$\text{Pow}(\overset{\checkmark}{LL}(k)) < \text{Pow}(\overset{\checkmark}{LR}(k)) < \underline{\text{Power}(\overset{\checkmark}{RBR})}$$

Error detection

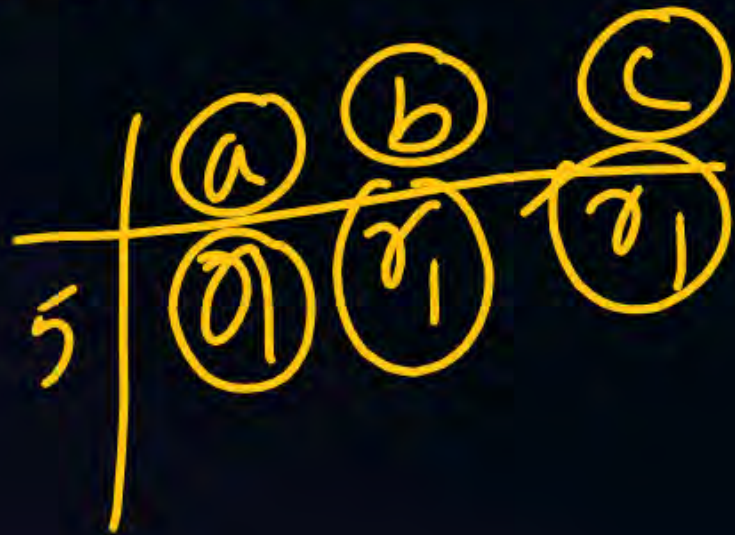
Predictive nature of TDP allows errors to be detected at the earliest possible time. The TDP parser stack can be used to repair the i/p as it contains information on what is expected to be seen in contrast with BUF where stack contains information about what has already been encountered in parsing.



→ LL(1) is simple TDP that can parse only a small class of grammars as there are restrictions on the grammar that it should be free from LR and it should be deterministic

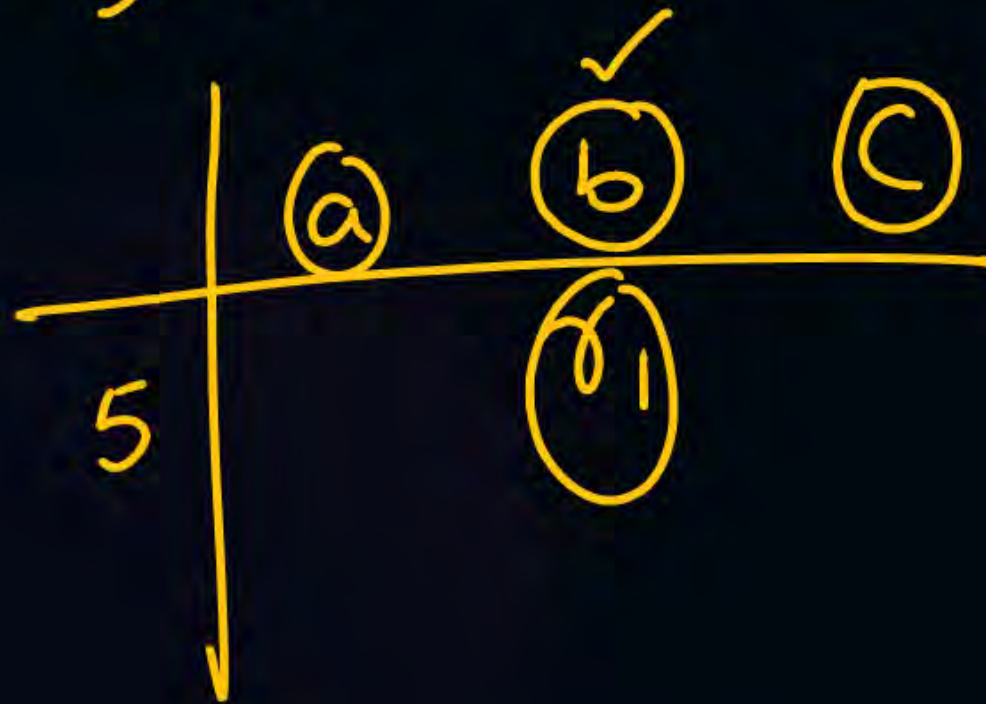
→ LR(0) is the simplest of all LR parsers but not used practically as it is less powerful. It does not use any look ahead

Therefore, it will parse only a small class of grammars.



SLR(1) is better than $\frac{LR(0)}{\downarrow}$ as it uses look ahead

SLR(1)



SLR(1) is not powerful like CLR(1)

CLR(1) is the most powerful parser but requires more table space

So LALR(1) is preferred as the table size is less than CLR(1)

And LALR(1) uses LR(1) items

1:30
2:30

3 min

Error recovery in LR parsers:-

An LR parser will detect end when it consults the parsing action table and find a blank entry. For each blank entry there will be a pointer to end recovery Subroutine. So that whenever the blank entry is referred, the corresponding Subroutine is called.

	id	+	*	(	)	\$
0	S ₃	<u>e₁</u>	<u>e₂</u>	S ₂	<u>e₃</u>	<u>e₄</u>

Parser Construction using ambiguous grammar:-

$$\begin{aligned}
 E &\rightarrow E + E \\
 &\quad / E * E \\
 &\quad / id
 \end{aligned}$$

$$\begin{aligned}
 E \\
 | \\
 id
 \end{aligned}$$

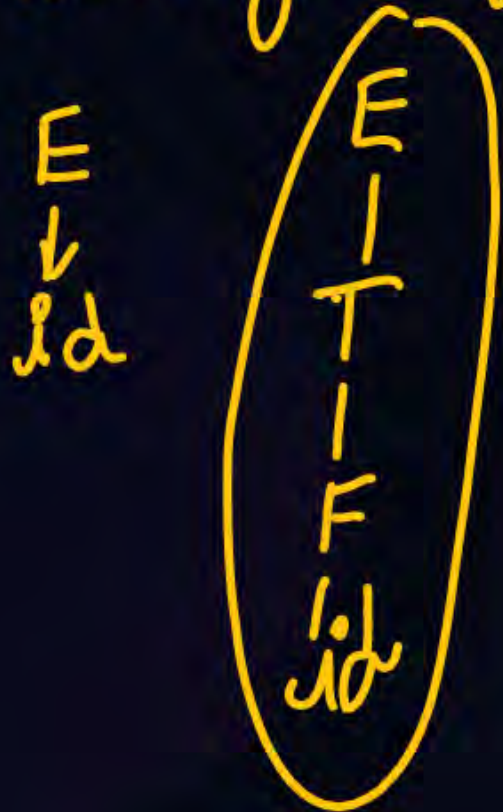
unambiguous
 $\Rightarrow$

$$\begin{aligned}
 E &\rightarrow E + T / T \\
 T &\rightarrow T * F / F \\
 F &\rightarrow id
 \end{aligned}$$

$$\begin{aligned}
 E \\
 | \\
 T \\
 | \\
 F \\
 | \\
 id
 \end{aligned}$$

Advantages of ambiguous grammars:-

- Ambiguous grammars are shorter and natural
- If the parser is constructed with ambiguous grammar, we can change precedence and associativity of the operators at a later time.
- No wastage of time





Let us construct SLR(1) parsing table for the ambiguous grammar

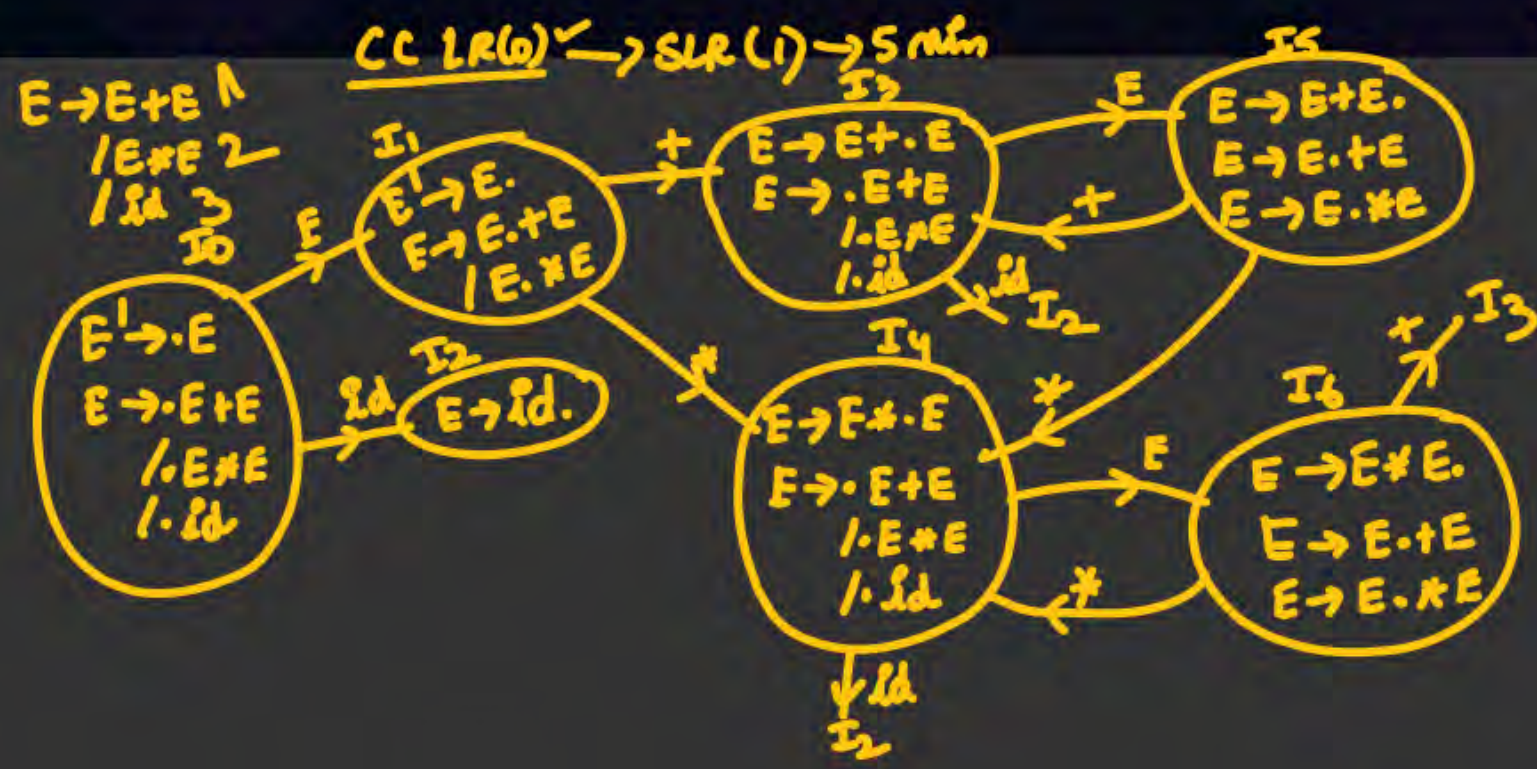
$E \rightarrow E + E$

$/ E * E$

$/ id$

We will get conflicts and if we can resolve the conflicts,

We can use the parsing table

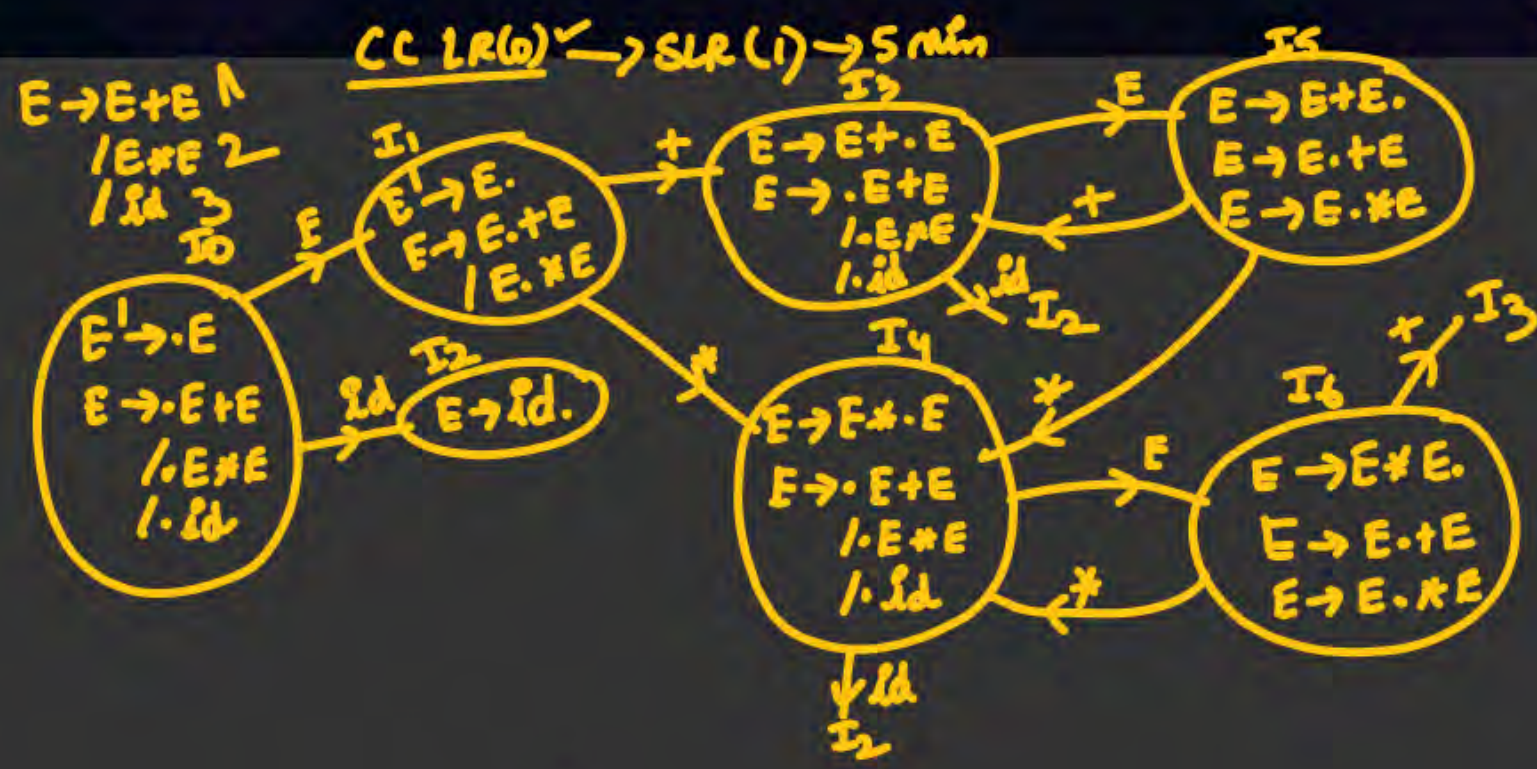


SLR(1):

		action			Goto
	id	+	*	\$	E
0	S ₂				1
1		S ₃	S ₄	acc	
2		r ₃	r ₃	r ₃	
3	S ₂				5
4	S ₂				6
5		S ₃ /r ₁	S ₄ /r ₁	r ₁	
6		S ₃ /r ₂	S ₄ /r ₂	r ₂	

Follow(E) = { \$, +, * } $I_6: E \rightarrow E * E.$
 r_2

$I_3: E \rightarrow E + E.$ r_1
 $I_3: E \rightarrow id.$ r_3
 r_3



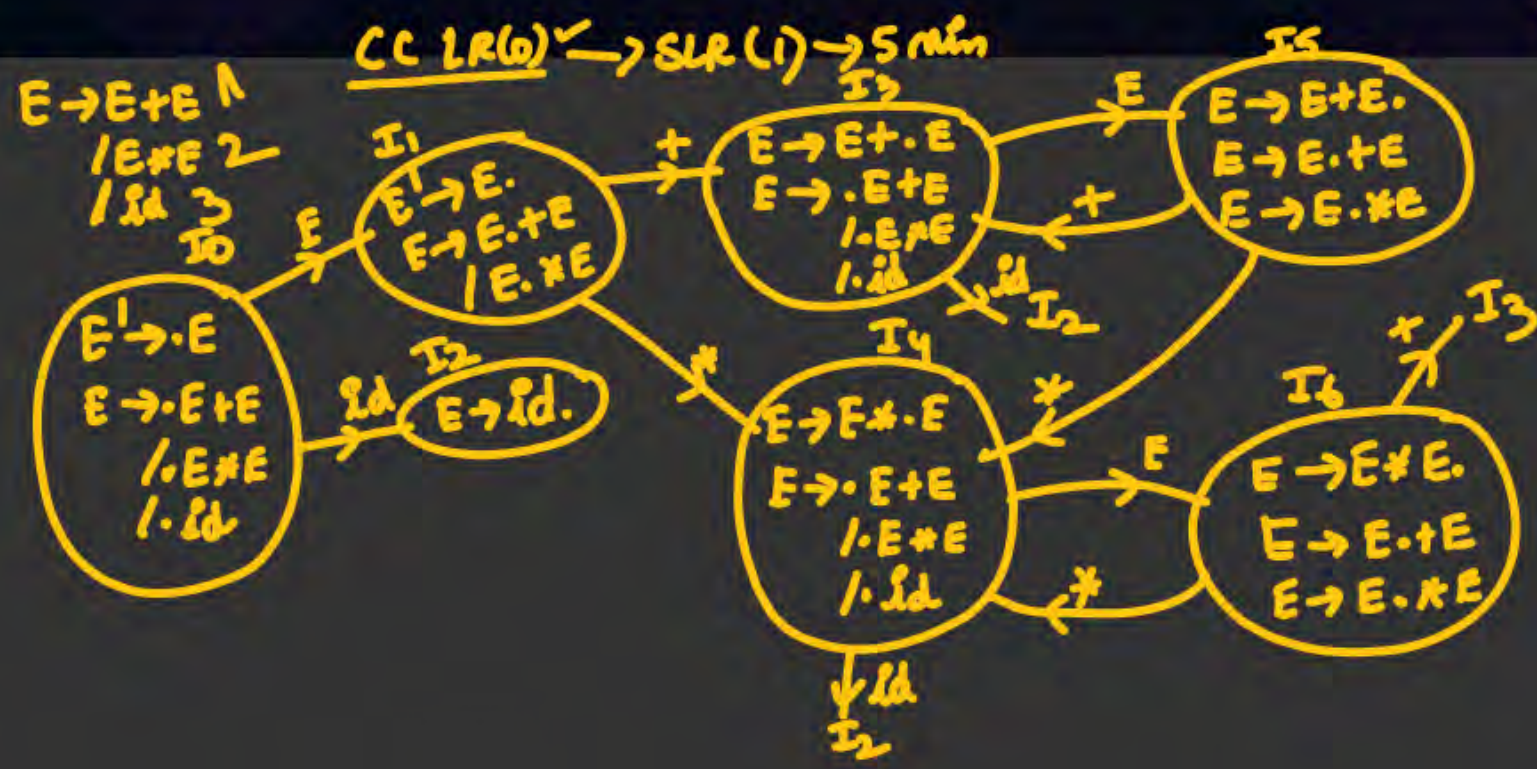
SLR(1):

	id	+	*	\$	Goto
0	S2				E
1		S3	S4	acc	
2		δ_3	δ_3	δ_3	
3	S2				5
4	S2				6
5		S3/ δ_1	S4/ δ_1	δ_1	
6		S3/ δ_2	S4/ δ_2	δ_2	

$S_3/\delta_1 \rightarrow \delta_1$

$E \rightarrow E + E \cdot$
 $E \rightarrow E \cdot + E$

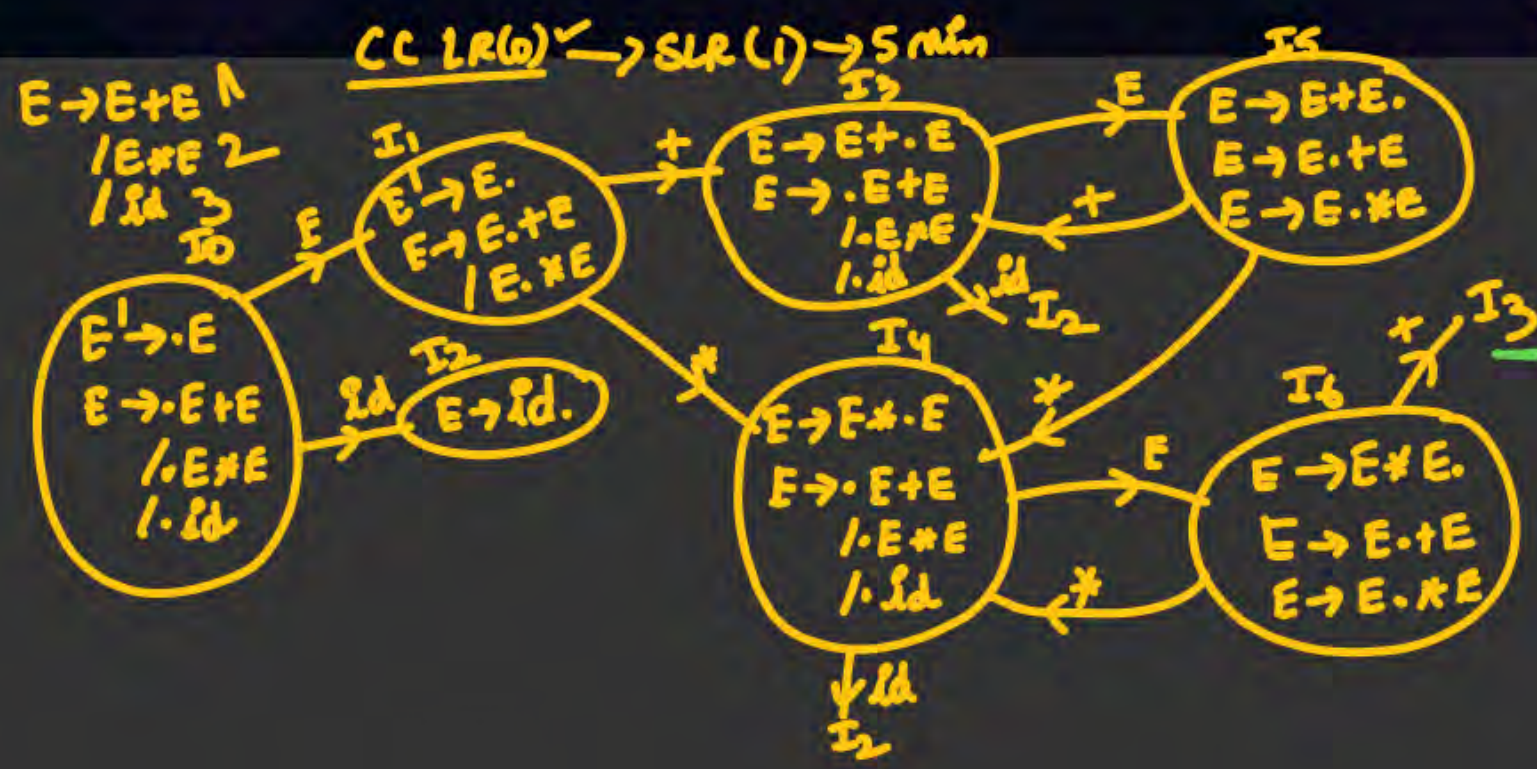
$id + id \cdot$ $+ id$



SLR(1):

		Action			Goto
	id	+	*	\$	E
0	S2				1
1		S3	S4	acc	
2		r3	r3	r3	
3	S2				5
4	S2				6
✓5		S3/r1	S4/r1	r1	
6		S3/r2	S4/r2	r2	

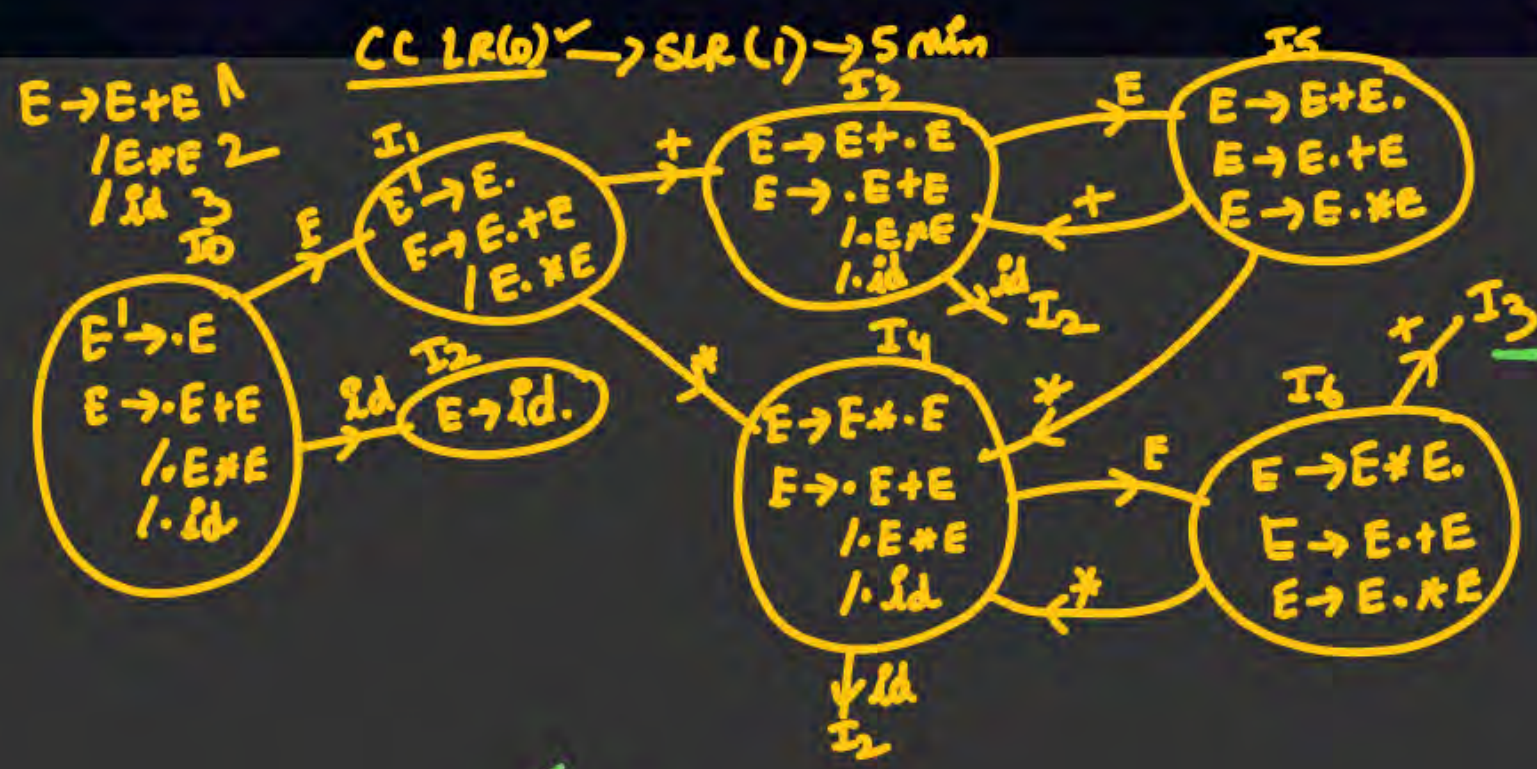
$S4/r1 \rightarrow S4$ $E \rightarrow E \cdot * E$ $S4$
 $E \rightarrow E + E \cdot$ $r1$
 $id + id \cdot * id$



$S_3/\delta_2 \rightarrow \delta_2$ $E \rightarrow E \cdot + E$ S_3
 $E \rightarrow E * E \cdot$
 $id * id \cdot + id$

SLR(1):

		Action			Goto
	id	+	*	\$	E
0	S_2				1
1		S_3	S_4	acc	
2		δ_3	δ_3	δ_3	
3	S_2				5
4	S_2				6
✓ 5		S_3/δ_1	S_4/δ_1	δ_1	
6		S_3/δ_2	S_4/δ_2	δ_2	



SLR(1):

		Action			Goto
	id	+	*	\$	E
0	S ₂				1
1		S ₃	S ₄	acc	
2		r ₃	r ₃	r ₃	
3	S ₂				5
4	S ₂				6
✓ 5		S ₃ /r ₁	S ₄ /r ₁	r ₁	
6		S ₃ /r ₂	S ₄ /r ₂	r ₂	

$S_4/r_2 \rightarrow r_2$

$E \rightarrow E \cdot * E$
 $E \rightarrow E * E \cdot$

Parm
SPT

id * id * id

We resolved all the conflicts

3 min

Syntax directed translation (SDT)



SDT is an extension of CFGs. This helps Compiler designer to translate the language Constructs directly by attaching Semantic actions or Sub-routines

Grammar + Semantic rules = SDT

With SDT, we can combine many tasks along with parsers.
Along with each production of the grammar, we attach a semantic rule.



Some examples of tasks performed by SDTs are:-

- 1) To store/retrieve type information in the Symbol table.
- 2) To perform consistency checks like type checking, parameter passing etc.
- 3) To issue error messages
- 4) To build Syntax trees
- 5) To generate intermediate or target codes

→ In SDT, we parse the i/p, Create parse tree
 and traverse the parse tree as needed to evaluate
 Semantic actions at the parse tree nodes

→ SDTs are also called attribute grammars

where

- 1) Each grammar symbol is assigned certain attributes
- 2) Each production is augmented with semantic rules which are used to define attribute values

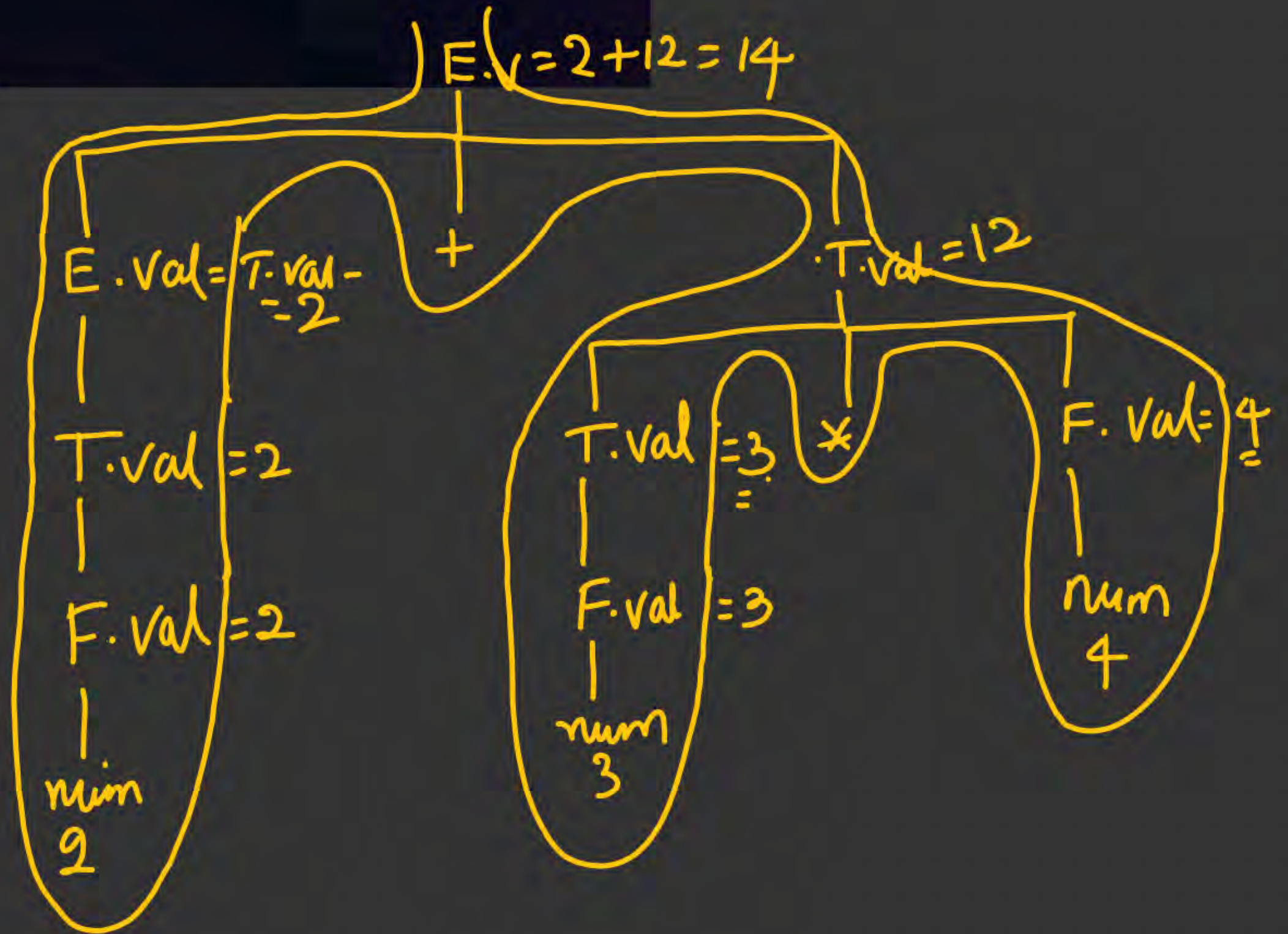
SDT for evaluation of expression:-

$E \rightarrow E_1 + T \{ E.val = E_1.val + T.val \}$
 $/ T \{ E.val = T.val \}$

$T \rightarrow T_1 * F \{ T.val = T_1.val * F.val \}$
 $/ F \{ T.val = F.val \}$

$F \rightarrow num \{ F.val = num.val \}$

$2 + 3 * 4$



Attributes of Grammar Symbols:

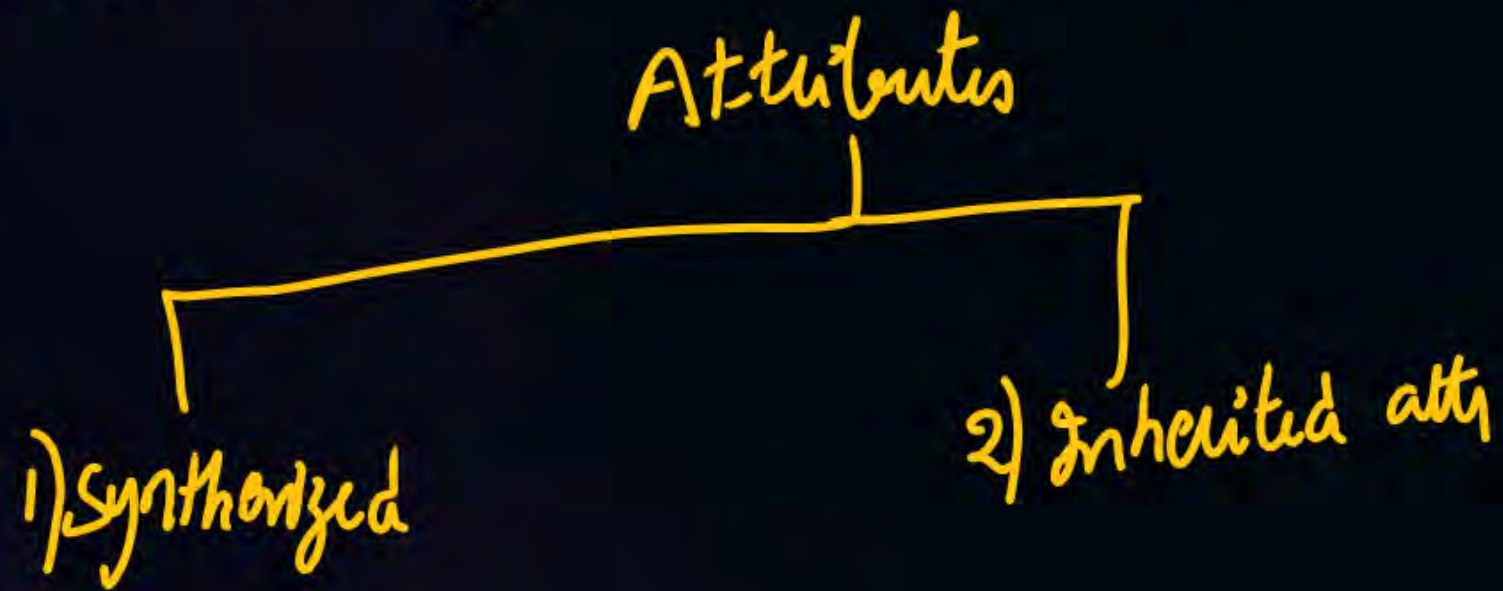
SDT is a generalization of a CFG in which each grammar symbol has an associated set of attributes. If we consider parse tree node as a record for holding information, then attribute is the name of each field in the record.



A grammar can have n attributes

The attributes for the grammar can be a type, value, address, pointer or a string. The attributes that can be associated with a grammar symbol can be classified into two types

Attributes:-



1) Synthesized attribute:- The value of a synthesized attribute at a node is computed from the values of the attributes at the children of that node in the parse tree

Ex:- $(A) \rightarrow (XYZ)$

Let us assume that there is an attribute 'S' with each grammar symbol. Then if it is evaluated as

$$\underline{A.S = f(X.S, Y.S, Z.S)}$$



THANK - YOU